

```
#import relevant libraries
import pandas as pd

# load data set into jupyter notebook
df_school = pd.read_csv("dim_schools.csv")
df_enrollments = pd.read_csv("fact_enrolments.csv")
df_performance = pd.read_csv("fact_performance.csv")
df_funding = pd.read_csv("raw_funding_requests.csv")
```

View data

df_school

	School_ID	School_Name	Region	Sector
0	2000	Centralbank College	Metro West	Government
1	2001	Southfield College	Coastal East	Independent
2	2002	Pacificside Tech	Coastal East	Catholic
3	2003	Forestview High	Metro North	Independent
4	2004	Sunnybank Tech	Regional West	Government
...	...	...	...	...
115	2115	Southfield Tech	Metro North	Government
116	2116	Pacificfield College	Metro West	Government
117	2117	Springview High	Regional West	Catholic
118	2118	Silverfield Grammar	Regional West	Government
119	2119	Eastview Academy	Metro North	Independent

[120 rows x 4 columns]

df_enrollments

	School_ID	Year	Enrolments_FTE	Teaching_Staff_FTE
0	2000	2022	1464	100
1	2000	2023	1492	97
2	2000	2024	1487	100
3	2001	2022	1006	84
4	2001	2023	1003	84
...	...	...	...	...
355	2118	2023	1381	91
356	2118	2024	1353	95
357	2119	2022	1082	90
358	2119	2023	1075	90
359	2119	2024	1081	90

[360 rows x 4 columns]

df_performance

	School_ID	Year	Avg_Attendance_Pct	NAPLAN_Score_Avg
0	2000	2022	88.9	526.0
1	2000	2023	90.3	528.0

2	2000	2024	92.2	533.0
3	2001	2022	95.2	541.0
4	2001	2023	94.0	530.0
...	...	...	...	...
355	2118	2023	79.9	527.0
356	2118	2024	82.9	519.0
357	2119	2022	97.0	538.0
358	2119	2023	95.6	535.0
359	2119	2024	96.3	547.0

[360 rows x 4 columns]

df_funding

	Request_ID	School_ID	Request_Date	Category	Status
Amount \					
0	REQ1000	2025	2022-02-03	Stem Lab	Declined
250000.0					
1	REQ1001	2066	2022-05-25	Capital Works	Pending
100000.0					
2	REQ1002	2025	2022-05-18	STEM Lab	Declined
50000.0					
3	REQ1003	2091	2023-04-06	IT Upgrade	Submitted
5000.0					
4	REQ1004	2061	2023-02-24	I.T. Upgrade	Declined
250000.0					
..	...	...	...	...	...
...					
107	REQ1096	2102	2022-04-05	Maintenance	Approved
5000.0					
108	REQ1097	2027	2023-11-29	IT Upgrade	Submitted
10000.0					
109	REQ1098	2081	2023-10-20	Stem Lab	Submitted
250000.0					
110	REQ1099	2109	2022-06-13	Cap Works	Approved
NaN					
111	REQ1099	2109	2022-06-13	Cap Works	Approved
NaN					

	Status_Date
0	2022-02-13
1	2022-05-29
2	2022-05-23
3	2023-04-11
4	2023-02-25
..	...
107	2022-04-10
108	2023-12-01
109	2023-10-24
110	2022-06-15

```
111 2022-06-30
```

```
[112 rows x 7 columns]
```

cleaning/processing df_school

```
df_school
```

	School_ID	School_Name	Region	Sector
0	2000	Centralbank College	Metro West	Government
1	2001	Southfield College	Coastal East	Independent
2	2002	Pacificside Tech	Coastal East	Catholic
3	2003	Forestview High	Metro North	Independent
4	2004	Sunnybank Tech	Regional West	Government
...	...	...	...	...
115	2115	Southfield Tech	Metro North	Government
116	2116	Pacificfield College	Metro West	Government
117	2117	Springview High	Regional West	Catholic
118	2118	Silverfield Grammar	Regional West	Government
119	2119	Eastview Academy	Metro North	Independent

```
[120 rows x 4 columns]
```

```
# df_school should have an unique school_id so we will check for  
duplicate school id
```

```
duplicate_rows = df_school[df_school.duplicated(subset=['School_ID'],  
keep=False)]
```

```
# no duplicate school ids which is good
```

```
duplicate_rows
```

```
Empty DataFrame
```

```
Columns: [School_ID, School_Name, Region, Sector]
```

```
Index: []
```

```
# check for blank rows
```

```
blank_rows= df_school[df_school.isna().any(axis=1)]
```

```
# no blank rows , which is good
```

```
blank_rows
```

```
Empty DataFrame
```

```
Columns: [School_ID, School_Name, Region, Sector]
```

```
Index: []
```

```
# look at region and sector columns for potential data standardization  
issues.
```

```
unique_Region = df_school['Region'].unique()
```

```
# no data standardisation issues in attribute region
```

```
unique_Region
```

```
array(['Metro West', 'Coastal East', 'Metro North', 'Regional West',
      'Metro South'], dtype=object)

unique_Sector = df_school['Sector'].unique()

# no data standardisation issues in school which is good
unique_Sector

array(['Government', 'Independent', 'Catholic'], dtype=object)
```

Cleaning/Processing df_enrollments

df_enrollments

	School_ID	Year	Enrolments_FTE	Teaching_Staff_FTE
0	2000	2022	1464	100
1	2000	2023	1492	97
2	2000	2024	1487	100
3	2001	2022	1006	84
4	2001	2023	1003	84
..	...	...	...	...
355	2118	2023	1381	91
356	2118	2024	1353	95
357	2119	2022	1082	90
358	2119	2023	1075	90
359	2119	2024	1081	90

[360 rows x 4 columns]

```
# df_enrollments should have no rows with the same school id and year
so let's check for that
duplicate_rows_enrollment =
df_enrollments[df_enrollments.duplicated(subset=['School_ID', 'Year'],
keep=False)]
```

```
# no duplicates which is good
duplicate_rows_enrollment
```

Empty DataFrame

Columns: [School_ID, Year, Enrolments_FTE, Teaching_Staff_FTE]

Index: []

Check max and min of Enrolments/Staff FTE to look for any outliers or suspicious numbers

```
max(df_enrollments['Enrolments_FTE'])
```

1735

```
min(df_enrollments['Enrolments_FTE'])
```

317

```

max(df_enrollments['Teaching_Staff_FTE'])
119
min(df_enrollments['Teaching_Staff_FTE'])
21
# check for blank rows
blank_rows_enrollment =
df_enrollments[df_enrollments.isna().any(axis=1)]
# no blanks found which is good
blank_rows_enrollment

Empty DataFrame
Columns: [School_ID, Year, Enrolments_FTE, Teaching_Staff_FTE]
Index: []

```

cleaning/processing of df_performance

```

df_performance

```

	School_ID	Year	Avg_Attendance_Pct	NAPLAN_Score_Avg
0	2000	2022	88.9	526.0
1	2000	2023	90.3	528.0
2	2000	2024	92.2	533.0
3	2001	2022	95.2	541.0
4	2001	2023	94.0	530.0
...	...	...	...	...
355	2118	2023	79.9	527.0
356	2118	2024	82.9	519.0
357	2119	2022	97.0	538.0
358	2119	2023	95.6	535.0
359	2119	2024	96.3	547.0

```

[360 rows x 4 columns]

# df_performance should have no rows with the same school id and year
so let's check for that
duplicate_rows_performance =
df_performance[df_performance.duplicated(subset=['School_ID', 'Year'],
keep=False)]

# no duplicates which is good
duplicate_rows_performance

Empty DataFrame
Columns: [School_ID, Year, Avg_Attendance_Pct, NAPLAN_Score_Avg]
Index: []

```

```
# check for blank rows
blank_rows_performance =
df_performance[df_performance.isna().any(axis=1)]

# we can see some schools for particular years are missing data. For a
# real task, this data quality issue may possibly be raised.
# For the purpose of this exercise since all schools are required to
# sit NAPLAN we will replace blank years with the schools average in the
# timeframe.
blank_rows_performance
```

	School_ID	Year	Avg_Attendance_Pct	NAPLAN_Score_Avg
5	2001	2024	94.2	NaN
10	2003	2023	97.8	NaN
27	2009	2022	91.6	NaN
64	2021	2023	90.6	NaN
67	2022	2023	90.8	NaN
87	2029	2022	91.5	NaN
91	2030	2023	88.7	NaN
98	2032	2024	91.4	NaN
117	2039	2022	83.2	NaN
167	2055	2024	81.5	NaN
200	2066	2024	92.9	NaN
215	2071	2024	92.8	NaN
216	2072	2022	91.7	NaN
218	2072	2024	92.1	NaN
230	2076	2024	89.5	NaN
239	2079	2024	82.6	NaN
262	2087	2023	91.3	NaN
268	2089	2023	92.1	NaN
297	2099	2022	80.5	NaN
305	2101	2024	92.2	NaN
330	2110	2022	96.3	NaN
341	2113	2024	95.1	NaN
347	2115	2024	91.8	NaN

```
# Replace blank Naplan scores with the mean of the School
df_performance['NAPLAN_Score_Avg'] =
df_performance['NAPLAN_Score_Avg'].fillna(df_performance.groupby('School_ID')['NAPLAN_Score_Avg'].transform('mean'))
```

```
# check for blank rows again
blank_rows_performance =
df_performance[df_performance.isna().any(axis=1)]
```

```
#blank rows have been replaced with averages.
blank_rows_performance
```

```
Empty DataFrame
Columns: [School_ID, Year, Avg_Attendance_Pct, NAPLAN_Score_Avg]
Index: []
```

```
# export cleaned performance data
df_performance.to_csv("fact_performance_cleaned.csv")
```

Check max and min of Avg_attendance_Pct/Npalan score to look for any outliers or suspicious numbers

```
max(df_performance['Avg_Attendance_Pct'])
98.0
min(df_performance['Avg_Attendance_Pct'])
78.3
max(df_performance['NAPLAN_Score_Avg'])
550.0
min(df_performance['NAPLAN_Score_Avg'])
480.0
```

Cleaning/Processing df_funding

```
df_funding
```

	Request_ID	School_ID	Request_Date	Category	Status
Amount \					
0	REQ1000	2025	2022-02-03	Stem Lab	Declined
250000.0					
1	REQ1001	2066	2022-05-25	Capital Works	Pending
100000.0					
2	REQ1002	2025	2022-05-18	STEM Lab	Declined
50000.0					
3	REQ1003	2091	2023-04-06	IT Upgrade	Submitted
5000.0					
4	REQ1004	2061	2023-02-24	I.T. Upgrade	Declined
250000.0					
...	...	...	...	...	...
...					
107	REQ1096	2102	2022-04-05	Maintenance	Approved
5000.0					
108	REQ1097	2027	2023-11-29	IT Upgrade	Submitted
10000.0					
109	REQ1098	2081	2023-10-20	Stem Lab	Submitted
250000.0					
110	REQ1099	2109	2022-06-13	Cap Works	Approved
NaN					
111	REQ1099	2109	2022-06-13	Cap Works	Approved
NaN					

```

      Status_Date
0      2022-02-13
1      2022-05-29
2      2022-05-23
3      2023-04-11
4      2023-02-25
..
107    2022-04-10
108    2023-12-01
109    2023-10-24
110    2022-06-15
111    2022-06-30

```

```
[112 rows x 7 columns]
```

```
# request id should be unique to ensure sum of amounts reflects total cost and there is no double counting.
```

```
duplicate_rows_funding =
df_funding[df_funding.duplicated(subset=['Request_ID'], keep=False)]
```

```
# there are duplicates so we need to fix this.
```

```
duplicate_rows_funding
```

	Request_ID	School_ID	Request_Date	Category	Status
Amount \					
39	REQ1039	2035	2023-08-11	Maintenance	Submitted
100000.0					
40	REQ1039	2035	2023-08-11	Maintenance	Approved
100000.0					
49	REQ1048	2026	2022-09-13	Sports Equip	Submitted
15000.0					
50	REQ1048	2026	2022-09-13	Sports Equip	Approved
15000.0					
54	REQ1052	2036	2022-03-21	Maintenance	Pending
15000.0					
55	REQ1052	2036	2022-03-21	Maintenance	Approved
15000.0					
67	REQ1064	2006	2023-05-28	IT Upgrade	Pending
NaN					
68	REQ1064	2006	2023-05-28	IT Upgrade	Approved
NaN					
72	REQ1068	2059	2022-07-15	Capital Works	Pending
100000.0					
73	REQ1068	2059	2022-07-15	Capital Works	Approved
100000.0					
74	REQ1069	2018	2023-02-22	IT Upgrade	Pending
5000.0					
75	REQ1069	2018	2023-02-22	IT Upgrade	Approved
5000.0					

79	REQ1073	2088	2023-10-21	Sports Equipment	Pending
5000.0					
80	REQ1073	2088	2023-10-21	Sports Equipment	Approved
5000.0					
81	REQ1074	2115	2022-04-21	Capital Works	Declined
100000.0					
82	REQ1074	2115	2022-04-21	Capital Works	Approved
100000.0					
91	REQ1083	2042	2023-06-26	IT Upgrade	Declined
10000.0					
92	REQ1083	2042	2023-06-26	IT Upgrade	Approved
10000.0					
100	REQ1091	2009	2022-12-02	Stem Lab	Declined
5000.0					
101	REQ1091	2009	2022-12-02	Stem Lab	Approved
5000.0					
104	REQ1094	2088	2022-11-02	STEM Lab	Submitted
50000.0					
105	REQ1094	2088	2022-11-02	STEM Lab	Approved
50000.0					
110	REQ1099	2109	2022-06-13	Cap Works	Approved
NaN					
111	REQ1099	2109	2022-06-13	Cap Works	Approved
NaN					

	Status_Date
39	2023-08-16
40	2023-08-31
49	2022-09-20
50	2022-09-25
54	2022-03-25
55	2022-04-06
67	2023-06-03
68	2023-06-08
72	2022-07-25
73	2022-07-27
74	2023-03-04
75	2023-03-05
79	2023-10-29
80	2023-11-10
81	2022-04-27
82	2022-05-08
91	2023-06-29
92	2023-07-16
100	2022-12-10
101	2022-12-22
104	2022-11-10
105	2022-11-18

```
110 2022-06-15
111 2022-06-30
```

```
# We will keep the row with the latest date, if a request ultimately
got approved
# the same request shouldnt also exist as declined. Therefore we will
take the most recent row and remove all other duplicates.
```

```
# we ll convert status date to date format , sort date by most recent,
then take first unique request ID
```

```
df_funding['Status_Date'] = pd.to_datetime(df_funding['Status_Date'])
df_funding_date_sorted = df_funding.sort_values(by='Status_Date',
ascending=False)
```

```
# sorted from most recent to least recent
df_funding_date_sorted
```

	Request_ID	School_ID	Request_Date	Category	Status
Amount \					
108	REQ1097	2027	2023-11-29	IT Upgrade	Submitted
10000.0					
42	REQ1041	2018	2023-11-24	Stem Lab	Submitted
15000.0					
35	REQ1035	2075	2023-11-18	Sports Equip	Approved
250000.0					
80	REQ1073	2088	2023-10-21	Sports Equipment	Approved
5000.0					
95	REQ1086	2101	2023-10-28	Maintenance	Pending
NaN					
..	...	...	...	...	...
...					
0	REQ1000	2025	2022-02-03	Stem Lab	Declined
250000.0					
14	REQ1014	2051	2022-01-29	STEM Lab	Declined
100000.0					
78	REQ1072	2069	2022-01-22	Sports Equip	Declined
100000.0					
62	REQ1059	2032	2022-01-08	Maintenance	Declined
50000.0					
59	REQ1056	2032	2022-01-09	Sports Equip	Declined
10000.0					

	Status_Date
108	2023-12-01
42	2023-11-30
35	2023-11-21
80	2023-11-10
95	2023-11-07
..	...
0	2022-02-13

```

14    2022-02-04
78    2022-02-01
62    2022-01-16
59    2022-01-14

```

```
[112 rows x 7 columns]
```

```
# Remove duplicates based on the 'Request_ID' column, keeping the first instance which would be the most recent
```

```
df_unique_funding =
df_funding_date_sorted.drop_duplicates(subset=['Request_ID'],
keep='first')
```

```
# cleaned request ID to avoid duplicate rows/double counting
```

```
df_unique_funding
```

	Request_ID	School_ID	Request_Date	Category	Status
Amount \					
108	REQ1097	2027	2023-11-29	IT Upgrade	Submitted
10000.0					
42	REQ1041	2018	2023-11-24	Stem Lab	Submitted
15000.0					
35	REQ1035	2075	2023-11-18	Sports Equip	Approved
250000.0					
80	REQ1073	2088	2023-10-21	Sports Equipment	Approved
5000.0					
95	REQ1086	2101	2023-10-28	Maintenance	Pending
NaN					
..	...	...	...	...	...
...					
0	REQ1000	2025	2022-02-03	Stem Lab	Declined
250000.0					
14	REQ1014	2051	2022-01-29	STEM Lab	Declined
100000.0					
78	REQ1072	2069	2022-01-22	Sports Equip	Declined
100000.0					
62	REQ1059	2032	2022-01-08	Maintenance	Declined
50000.0					
59	REQ1056	2032	2022-01-09	Sports Equip	Declined
10000.0					

	Status_Date
108	2023-12-01
42	2023-11-30
35	2023-11-21
80	2023-11-10
95	2023-11-07
..	...
0	2022-02-13
14	2022-02-04

```
78    2022-02-01
62    2022-01-16
59    2022-01-14
```

```
[100 rows x 7 columns]
```

```
# check for blank rows
```

```
blank_rows_funding =  
df_unique_funding[df_unique_funding.isna().any(axis=1)]
```

```
# there exists blank rows for amount which may possibly lead to  
inaccurate calculations. Normally, it would be ideal if the data  
# quality issue could be addressed and a number is given. However for  
the purpose of this task we can take the average of the relevant  
category
```

```
# once we standardise the category.
```

```
blank_rows_funding
```

	Request_ID	School_ID	Request_Date	Category	Status
Amount \					
95	REQ1086	2101	2023-10-28	Maintenance	Pending
NaN					
93	REQ1084	2034	2023-10-02	Sports Equip	Pending
NaN					
18	REQ1018	2009	2023-06-17	STEM Lab	Pending
NaN					
68	REQ1064	2006	2023-05-28	IT Upgrade	Approved
NaN					
60	REQ1057	2040	2023-04-03	Sports Equipment	Declined
NaN					
48	REQ1047	2003	2022-12-17	Maintenance	Approved
NaN					
31	REQ1031	2060	2022-11-25	I.T. Upgrade	Approved
NaN					
17	REQ1017	2089	2022-10-25	I.T. Upgrade	Declined
NaN					
106	REQ1095	2025	2022-08-22	Cap Works	Submitted
NaN					
111	REQ1099	2109	2022-06-13	Cap Works	Approved
NaN					

	Status_Date
95	2023-11-07
93	2023-10-11
18	2023-06-24
68	2023-06-08
60	2023-04-05
48	2022-12-27
31	2022-11-30
17	2022-10-30

```
106 2022-08-24
111 2022-06-30
```

```
# look at Category column for potential data standardization issues.
unique_Category = df_unique_funding['Category'].unique()
```

```
# we can see a lot of equivalent categories but are named
differently , this needs to be fixed or will cause standardization
issues when aggregating
# by category later on.
unique_Category
```

```
array(['IT Upgrade', 'Stem Lab', 'Sports Equip', 'Sports Equipment',
      'Maintenance', 'Cap Works', 'Capital Works', 'I.T. Upgrade',
      'STEM Lab'], dtype=object)
```

```
# We'll replace some words to standardize the attribute.
```

```
# replace 'I.T. Upgrade' with 'IT Upgrade'
df_unique_funding['Category'] =
df_unique_funding['Category'].replace('I.T. Upgrade', 'IT Upgrade')
```

```
# replace 'Stem Lab' with 'STEM Lab'
df_unique_funding['Category'] =
df_unique_funding['Category'].replace('Stem Lab', 'STEM Lab')
```

```
# replace 'Sports Equip' with 'Sports Equipment'
df_unique_funding['Category'] =
df_unique_funding['Category'].replace('Sports Equip', 'Sports
Equipment')
```

```
# replace 'Cap Works' with 'Capital Works'
df_unique_funding['Category'] =
df_unique_funding['Category'].replace('Cap Works', 'Capital Works')
```

```
C:\Users\Angel\AppData\Local\Temp\ipykernel_6636\1665559170.py:4:
SettingWithCopyWarning:
```

```
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation:
```

```
https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#
returning-a-view-versus-a-copy
```

```
df_unique_funding['Category'] =
df_unique_funding['Category'].replace('I.T. Upgrade', 'IT Upgrade')
```

```
C:\Users\Angel\AppData\Local\Temp\ipykernel_6636\1665559170.py:7:
SettingWithCopyWarning:
```

```
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation:
```

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df_unique_funding['Category'] =  
df_unique_funding['Category'].replace('Stem Lab', 'STEM Lab')  
C:\Users\Angel\AppData\Local\Temp\ipykernel_6636\1665559170.py:10:  
SettingWithCopyWarning:  
A value is trying to be set on a copy of a slice from a DataFrame.  
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df_unique_funding['Category'] =  
df_unique_funding['Category'].replace('Sports Equip', 'Sports  
Equipment')  
C:\Users\Angel\AppData\Local\Temp\ipykernel_6636\1665559170.py:13:  
SettingWithCopyWarning:  
A value is trying to be set on a copy of a slice from a DataFrame.  
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df_unique_funding['Category'] =  
df_unique_funding['Category'].replace('Cap Works', 'Capital Works')
```

check again now that we replaced certain values.

```
unique_Category = df_unique_funding['Category'].unique()
```

we can see category is now standardized properly

```
unique_Category
```

```
array(['IT Upgrade', 'STEM Lab', 'Sports Equipment', 'Maintenance',  
      'Capital Works'], dtype=object)
```

Replace blank amounts with the mean of their category

```
df_unique_funding['Amount'] =  
df_unique_funding['Amount'].fillna(df_unique_funding.groupby('Category  

```

C:\Users\Angel\AppData\Local\Temp\ipykernel_6636\3674558141.py:2:

SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df_unique_funding['Amount'] =  
df_unique_funding['Amount'].fillna(df_unique_funding.groupby('Category  

```

```
# check for blank rows again
blank_rows_funding =
df_unique_funding[df_unique_funding.isna().any(axis=1)]

# None as we have chosen to replace blanks with aggregated mean by
category
blank_rows_funding

Empty DataFrame
Columns: [Request_ID, School_ID, Request_Date, Category, Status,
Amount, Status_Date]
Index: []
```

```
df_unique_funding
```

	Request_ID	School_ID	Request_Date	Category	Status	\
108	REQ1097	2027	2023-11-29	IT Upgrade	Submitted	
42	REQ1041	2018	2023-11-24	STEM Lab	Submitted	
35	REQ1035	2075	2023-11-18	Sports Equipment	Approved	
80	REQ1073	2088	2023-10-21	Sports Equipment	Approved	
95	REQ1086	2101	2023-10-28	Maintenance	Pending	
..	...	...	...	...	...	
0	REQ1000	2025	2022-02-03	STEM Lab	Declined	
14	REQ1014	2051	2022-01-29	STEM Lab	Declined	
78	REQ1072	2069	2022-01-22	Sports Equipment	Declined	
62	REQ1059	2032	2022-01-08	Maintenance	Declined	
59	REQ1056	2032	2022-01-09	Sports Equipment	Declined	

	Amount	Status_Date
108	10000.000000	2023-12-01
42	15000.000000	2023-11-30
35	250000.000000	2023-11-21
80	5000.000000	2023-11-10
95	53076.923077	2023-11-07
..	...	...
0	250000.000000	2022-02-13
14	100000.000000	2022-02-04
78	100000.000000	2022-02-01
62	50000.000000	2022-01-16
59	10000.000000	2022-01-14

```
[100 rows x 7 columns]
```

Look for potential outliers in amount or suspicious numbers

```
max(df_unique_funding['Amount'])

250000.0

min(df_unique_funding['Amount'])
```

5000.0

#export cleaned funding data

df_unique_funding.to_csv("cleaned_funding_requests.csv")